

Practical No.:	01
Practical Title :	Study and Implementation of Machine Learning Workflow with Model Evaluation, Validation, and Selection Techniques
Aim:	To study and implement the complete Machine Learning (ML) workflow, including data preprocessing, train-test splitting, model training, cross-validation, evaluation using statistical metrics, and final model selection.
Objectives:	1. Understand the end-to-end Machine Learning model development lifecycle. 2. Implement data preprocessing and feature scaling on raw datasets. 3. Split datasets using train-validation-test strategies and <i>K</i>-Fold Cross-Validation. 4. Analyze model performance using evaluation metrics (MAE, RMSE, R^2, Accuracy, Precision, Recall, F1-Score).

Theory:**1. The Machine Learning Workflow**

Developing a reliable Machine Learning model requires a structured workflow to ensure that the model generalizes well to unseen real-world data:

- Stage 1: Raw Data
- Stage 2: Data Preprocessing
- Stage 3: Data Splitting
- Stage 4: Model Training
- Stage 5: Model Validation
- Stage 6: Evaluation & Selection generate this image

1. **Data Preprocessing & Feature Engineering:** Handling missing values, scaling continuous features (e.g., StandardScaler, MinMaxScaler), and encoding categorical labels.
2. **Data Splitting:** Partitioning data into training, validation, and test subsets to avoid data leakage and evaluate generalization capacity.
3. **Model Training:** Fitting baseline and candidate machine learning algorithms on training data.
4. **Validation & Hyperparameter Tuning:** Adjusting model parameters using *K*-Fold Cross-Validation to optimize performance without overfitting.

5. **Model Evaluation & Selection:** Testing candidate models against quantitative evaluation metrics to pick the top-performing architecture.

2. Data Splitting Strategies & Cross-Validation

To measure how well a model generalizes, the dataset must be split correctly:

- **Train / Test Split:** Typically an 80/20 or 70/30 division. The model learns from the training set and is evaluated on the independent test set.
- **K-Fold Cross-Validation:** The dataset is split into K equal-sized subsets (folds). The model is trained on $K - 1$ folds and validated on the remaining fold. This process repeats K times, and the average score provides an unbiased performance estimate.
- **Stratified K-Fold:** A variation of K -Fold used for imbalanced classification tasks to ensure each fold maintains the same percentage of samples for each target class.

3. Evaluation Metrics

A. Regression Metrics

When predicting continuous variables (e.g., price, temperature):

- **Mean Absolute Error (MAE):** Average magnitude of errors without considering direction.

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE):** Penalizes larger errors heavier than MAE.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

- **Root Mean Squared Error (RMSE):** Interpretable in the same unit as the target variable.

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}$$

- **Coefficient of Determination (R^2 Score):** Measures how well the model explains the variance of the target variable (scale from 0 to 1).

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

B. Classification Metrics

When predicting categorical outcomes:

- **Accuracy:** Overall ratio of correct predictions.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Ratio of true positive predictions to total predicted positives.

$$\text{Precision} = \frac{TP}{TP + FP}$$

- **Recall (Sensitivity):** Ratio of true positive predictions to actual positive instances.

$$\text{Recall} = \frac{TP}{TP + FN}$$

- **F1-Score:** Harmonic mean of Precision and Recall.

$$\text{F1-Score} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

4. Model Selection Techniques

Model selection involves comparing candidate algorithms (e.g., Linear Regression, Decision Trees, Random Forests, Support Vector Machines) using identical validation pipelines. The algorithm with the highest cross-validated score and lowest generalization error on test data is chosen for deployment.

Conclusion: We successfully implemented the Machine Learning workflow by preprocessing data, using K -fold cross-validation to ensure unbiased results, and selecting the optimal model based on statistical performance metrics.

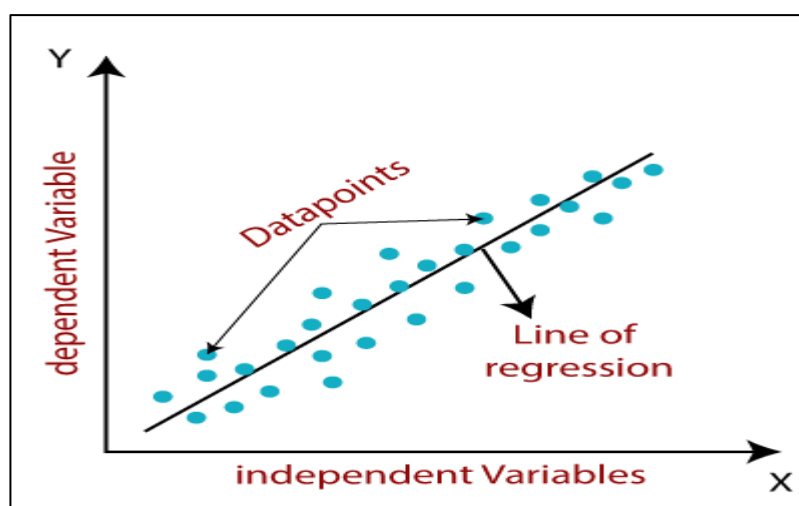
Practical No.:	02
Practical Title :	To develop a Linear Regression model
Aim:	To study and implement the Machine Learning workflow including data preprocessing, dataset splitting, model training, model evaluation, validation, and selection of the best-performing machine learning model.
Objectives:	1. Understand the basic Machine Learning workflow. 2. Understand the different steps involved in developing an ML model. 3. Perform data preprocessing before model training. 4. Divide a dataset into training and testing sets.

Theory:

1. Introduction to Linear Regression

Linear Regression is one of the simplest and most popular supervised Machine Learning algorithms used for **predictive analysis**. It predicts continuous or numeric output variables—such as sales, salary, age, or product prices—based on input features.

The main goal of linear regression is to find a linear relationship between a dependent target variable (y) and one or more independent predictor variables (x), represented visually as a **line of best fit** (or line of regression).



2. Types of Linear Regression

A. Simple Linear Regression

Simple Linear Regression is used when there is only one independent variable (x) to predict the dependent variable (y).

- Equation:

$$y = a_0 + a_1x$$

- Components:
 - y = Dependent / Target variable
 - x = Independent / Feature variable
 - a_0 = y -intercept (the value of y when $x = 0$)
 - a_1 = Slope / Coefficient (measures how much y changes per unit change in x)

B. Multiple Linear Regression

Multiple Linear Regression is used when there are two or more independent variables ($x_1, x_2, \dots, x_n$) predicting the dependent variable (y).

- Equation:

$$y = a_0 + a_1x_1 + a_2x_2 + a_3x_3 + \dots + a_nx_n$$

- Components:
 - y = Dependent / Target variable
 - $x_1, x_2, \dots, x_n$ = Independent feature variables
 - a_0 = Intercept
 - $a_1, a_2, \dots, a_n$ = Slopes / Coefficients corresponding to each input feature

3. Steps to Build a Linear Regression Model

1. Data Preprocessing: Handle missing data, scale numerical features, and remove extreme outliers.
2. Train-Test Splitting: Divide the dataset into training (e.g., 80%) and testing (e.g., 20%) subsets.
3. Model Training: Fit the linear regression model on the training set to calculate the optimal intercept (a_0) and slopes ($a_1, a_2, \dots$).
4. Prediction & Evaluation: Pass the test data into the trained model and assess performance using evaluation metrics like Mean Squared Error (MSE) and R^2 Score.

Conclusion:

We successfully implemented a simple linear regression model to analyze and quantify the relationship between a dependent and an independent variable. The analysis illustrated how a best-fit straight line can be mathematically derived to predict continuous outcomes based on input data.

Practical No.:	03
Practical Title :	To implement Logistic Regression.
Aim:	To implement Logistic Regression on a given dataset and perform classification of data into predefined classes.
Objectives:	• To understand the concept of supervised learning by estimating the probability of class membership using Logistic Regression. • To classify data into different categories by applying the sigmoid function and decision boundary.

Theory:

1. What is Logistic Regression?

Logistic Regression is a supervised machine learning algorithm used for **classification tasks** where the target output variable is categorical (e.g., 0 or 1, Yes or No, Pass or Fail).

While Linear Regression predicts continuous numerical outputs, Logistic Regression predicts the **probability** that a given input data point belongs to a specific category.

2. How Does It Work? (The Sigmoid Function)

To map input values into a valid probability range between 0 and 1, Logistic Regression uses an S-shaped curve called the **Sigmoid (or Logistic) Function**:

Where:

- $z = a_0 + a_1x_1 + a_2x_2 + \dots + a_nx_n$ (the linear combination of input features and weights)
- e = Euler's constant (≈ 2.718)
- $\sigma(z)$ = Output probability between 0.0 and 1.0

3. Decision Threshold

To convert the calculated probability into a final discrete class label, a **decision threshold** (commonly set at 0.5) is applied:

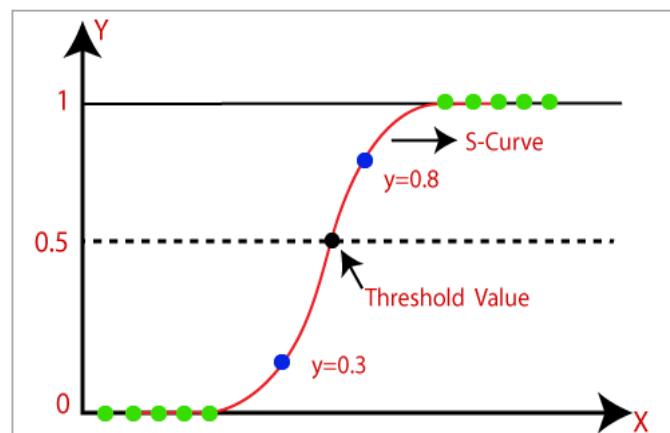
- **If Probability ≥ 0.5 :** Class 1 (e.g., Positive / Pass / Fraud)
- **If Probability < 0.5 :** Class 0 (e.g., Negative / Fail / Legitimate)

4. Types of Logistic Regression

1. **Binary Logistic Regression:** The target variable has only two possible classes (e.g., 0 or 1).
2. **Multinomial Logistic Regression:** The target variable has three or more unordered classes (e.g., Apple, Banana, Orange).
3. **Ordinal Logistic Regression:** The target variable has ordered categories (e.g., Low, Medium, High).

5. Assumptions of Logistic Regression

- The target variable must be categorical.
- The input features should be independent of each other (low or no multicollinearity).
- There should be a linear relationship between the independent variables and the log-odds of the outcome.



Conclusion:

We successfully implemented a logistic regression model to perform binary classification on the provided dataset. The experiment demonstrated how a linear combination of features can be transformed into a probability score using the Sigmoid (Logistic) function, allowing for precise categorical predictions.

Practical No.:	04
Practical Title :	To develop a Decision Tree classification model using the Gini Index
Aim:	To study, construct, and evaluate a Decision Tree classification model using the Gini Index criterion for decision splits, encompassing the complete workflow from data cleaning to model visualization and performance assessment
Objectives:	1. Perform essential data cleaning, handling missing values, and executing Exploratory Data Analysis (EDA). 2. Apply data transformation techniques such as feature encoding and scaling. 3. Understand the concept of decision tree splitting using the Gini Index. 4. Train and visualize a Decision Tree Classifier model.

Theory:

1. Introduction to Decision Trees

A Decision Tree is a popular supervised machine learning algorithm used for both classification and regression tasks. It structures decisions in a tree-like flow consisting of:

- Root Node: Represents the entire dataset, which gets split into subsets.
- Internal Nodes: Represent feature attribute tests/decisions.
- Branches: Represent the outcome of a decision rule.
- Leaf Nodes (Terminal Nodes): Represent the final class labels or predictions.

2. The Gini Index (Gini Impurity)

The Gini Index measures the probability of a randomly chosen element being incorrectly classified if it was randomly labeled according to the distribution of labels in the dataset. It ranges from 0 (pure node) to 0.5 (maximum impurity for binary classification).

$$\text{Gini Index} = 1 - \sum_{i=1}^C (p_i)^2$$

Where:

- C = Number of total target classes.
- p_i = Probability of an item belonging to class i .

Splitting Rule: The Decision Tree algorithm evaluates all candidate features and chooses the

feature split that yields the lowest weighted Gini Index (highest purity gain).

3. Machine Learning Workflow Steps

1. Data Cleaning: Identifying and handling null values, duplicate records, and outliers.
2. Exploratory Data Analysis (EDA): Visualizing feature distributions, correlations, and target class balance.
3. Data Transformation: Converting categorical strings into numeric values (Label Encoding / One-Hot Encoding) and splitting data into train-test sets.
4. Model Training: Fitting the Decision Tree Classifier using `criterion='gini'`.
5. Decision Tree Visualization: Rendering the learned tree structure to interpret feature split rules visually.
6. Performance Evaluation: Calculating metrics using the test set predictions.

4. Overfitting and Tree Pruning

Decision trees easily overfit by growing deep branches that memorize noise in the training set.

To control overfitting:

- Pre-Pruning: Setting parameters such as `max_depth`, `min_samples_split`, or `min_samples_leaf` before training to limit tree growth.
- Post-Pruning: Removing insignificant nodes after growing the full tree.

5. Evaluation Metrics

To assess performance, the model predictions are mapped into a Confusion Matrix to compute:

- Accuracy: Overall ratio of correct classifications.
- Precision: Accuracy of positive class predictions.
- Recall: Proportion of actual positive cases successfully caught.
- F1-Score: The harmonic mean of Precision and Recall.

Conclusion:

We successfully implemented an IoT system to collect sensor data in real-time and send it to the cloud for remote monitoring and automated control.

Practical No.:	05
Practical Title :	To implement CART decision tree algorithm
Aim:	To implement the CART (Classification and Regression Tree) algorithm for building a decision tree model and use it for classification or regression tasks.
Objective:	To understand and implement the CART decision tree algorithm for building an efficient model for classification and regression tasks.

Theory:

1. Introduction to CART

CART stands for **Classification and Regression Trees**. It is a popular supervised machine learning algorithm introduced by Leo Breiman in 1984.

Unlike algorithms that only perform classification, CART can handle:

- **Categorical target variables:** Predicts class labels (Classification).
- **Continuous numerical targets:** Predicts numeric values (Regression).

A key characteristic of CART is that it always constructs **Strictly Binary Trees**—meaning every internal node splits into exactly **two branches** (e.g., Yes/No or $\leq x / > x$).

2. Splitting Criteria in CART

CART selects the best feature to split a node based on the target variable type:

A. For Classification Trees (Categorical Data)

CART uses the **Gini Index (Gini Impurity)** to measure node impurity. It selects the feature split that results in the lowest Gini score.

$$\text{Gini Index} = 1 - \sum_{i=1}^c (p_i)^2$$

- **Gini = 0:** Completely pure node (all items belong to one class).
- **Gini = 0.5:** Maximum impurity for binary classification.

B. For Regression Trees (Continuous Data)

CART uses **Mean Squared Error (MSE)** or **Variance Reduction** to determine splits. It picks the split that minimizes the squared differences from the node mean.

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y})^2$$

Where $\hat{y}$ is the average target value of all data points in that node.

3. Making Predictions at Leaf Nodes

- **Classification:** Assigns the **majority class label** among the training samples in that leaf node.
- **Regression:** Predicts the **average (mean) value** of the training samples in that leaf node.

4. Preventing Overfitting in CART

To keep the tree from over-expanding and memorizing training noise, CART uses two main strategies:

1. **Pre-Pruning:** Setting parameters like maximum tree depth (max_depth) or minimum samples per leaf (min_samples_leaf).
2. **Post-Pruning (Cost-Complexity Pruning - c_p):** Growing a full tree first and then trimming weak branches that contribute little to overall accuracy.

5. Advantages of CART

- Handles both continuous and categorical data naturally.
- Works well with missing values and noisy data.
- Easy to interpret and visualize as visual flowcharts.
- Robust against outliers due to rule-based splitting.

Conclusion:

We successfully implemented and evaluated the **CART (Classification and Regression Tree)** algorithm. By utilizing the Gini Index for feature splits, we constructed a binary decision tree that effectively classified data points based on feature rules, achieving optimal accuracy on test data.

Practical No.:	06
Practical Title :	Implementation of Ensemble Learning Methods
Aim:	To implement Ensemble Learning methods using data cleaning, data transformation, model training, and performance evaluation.
Objective:	To understand and implement Ensemble Learning by performing data cleaning, exploratory data analysis, data transformation, model training, and performance evaluation to compare different ensemble methods and select the best-performing model.

Theory:

1. Ensemble Learning

Ensemble Learning is a Machine Learning technique that combines multiple models to improve prediction accuracy and reliability. Instead of depending on a single model, it combines the predictions of several models to reduce errors and improve generalization.

2. Data Cleaning

Data cleaning is the process of preparing raw data by handling missing values, removing duplicate records, correcting inconsistent values, and dealing with noisy or unwanted data. Clean data helps improve the quality and performance of Machine Learning models.

3. Exploratory Data Analysis (EDA)

EDA is used to understand the structure and characteristics of a dataset. It includes statistical analysis and visualization to identify patterns, distributions, relationships, correlations, and outliers. Common visualizations include histograms, box plots, scatter plots, and correlation heatmaps.

4. Data Transformation

Data transformation converts cleaned data into a suitable format for Machine Learning. It includes encoding categorical variables, scaling numerical features, feature selection, and splitting the dataset into training and testing sets.

5. Ensemble Methods

The major Ensemble Learning methods include **Bagging, Random Forest, and Boosting**. Bagging trains multiple models independently and combines their predictions. Random Forest combines multiple Decision Trees using random samples and features. Boosting trains models sequentially, where each model focuses on correcting the errors of previous models.

6. Model Training and Evaluation

The transformed training data is used to train the ensemble models. Their performance is evaluated using suitable metrics such as **Accuracy, Precision, Recall, and F1-Score** for classification and **MAE, RMSE, and R² Score** for regression. The models are compared to select the best-performing model.

Conclusion

We successfully implemented Ensemble Learning by performing data cleaning, Exploratory Data Analysis, data transformation, model training, and performance evaluation. Different ensemble methods were studied and compared using suitable evaluation metrics. The practical demonstrated that combining multiple models can improve prediction accuracy, reduce errors, and provide more reliable results. The best-performing model can be selected based on its evaluation performance and ability to generalize to unseen data.

Practical No.:	07
Practical Title :	Implementation of Bayesian Classification Model
Aim:	To implement a Bayesian Classification model by performing data cleaning, exploratory data analysis, data transformation, model training, and performance evaluation.
Objective:	To understand and apply Bayesian classification principles by preparing the dataset, analyzing its characteristics, transforming the data, training a Bayesian classification model, and evaluating its performance using suitable classification metrics..

Theory:

1. Bayesian Classification

Bayesian Classification is a supervised Machine Learning technique based on **Bayes' Theorem**. It calculates the probability of a class based on the given input features and assigns the observation to the class with the highest probability.

2. Data Cleaning

Data cleaning involves handling missing values, removing duplicate records, correcting inconsistent data, and dealing with unwanted or noisy observations. This improves the quality of the dataset before model training.

3. Exploratory Data Analysis (EDA)

EDA is used to understand the dataset by examining its structure, statistical properties, feature distributions, relationships, and possible outliers. Graphs and statistical summaries help identify important patterns in the data.

4. Data Transformation

Data transformation prepares the dataset for classification. It may include encoding categorical features, scaling numerical features, selecting relevant features, and splitting the dataset into training and testing sets.

5. Bayes' Theorem

Bayesian classification is based on Bayes' Theorem, which determines the probability of a hypothesis or class given the available evidence. It is represented as:

$$P(C|X) = [P(X|C) \times P(C)] / P(X)$$

where $P(C|X)$ is the posterior probability, $P(X|C)$ is the likelihood, $P(C)$ is the prior probability, and $P(X)$ is the evidence.

6. Naive Bayes Classifier

Naive Bayes is a commonly used Bayesian classification algorithm. It assumes that the input features are conditionally independent given the class. Despite this assumption, it performs effectively on many classification problems and is commonly used for applications such as text classification and spam detection.

7. Model Training and Evaluation

The transformed training data is used to train the Bayesian classifier. The trained model is then tested on unseen data. Its performance can be evaluated using **Accuracy, Precision, Recall, and F1-Score**. These metrics help determine the effectiveness of the classification model.

Conclusion

We successfully implemented a Bayesian Classification model by performing data cleaning, exploratory data analysis, data transformation, model training, and performance evaluation. The Bayesian classifier was evaluated using suitable classification metrics, demonstrating the application of Bayesian learning principles for classification problems.

Practical No.:	08
Practical Title :	Implementation of Naive Bayes Classifier and Bayesian Network
Aim:	To implement Naive Bayes Classifier and Bayesian Network for classification and evaluate their performance using suitable metrics.
Objective:	To understand and apply Bayesian learning principles by implementing Naive Bayes and Bayesian Network models for classification, training them on a prepared dataset, and evaluating their classification performance.

Theory:

1. Naive Bayes Classifier

Naive Bayes is a supervised Machine Learning classification algorithm based on **Bayes' Theorem**. It predicts the class of an observation by calculating the probability of each possible class from the given features. It assumes that the features are conditionally independent given the class. Naive Bayes is simple, fast, and commonly used for text classification, spam detection, and other classification problems.

2. Bayes' Theorem

Bayes' Theorem is used to calculate the probability of a class based on the available evidence.

$$P(C|X) = \frac{P(X|C) \times P(C)}{P(X)}$$

Where **P(C|X)** is the posterior probability, **P(X|C)** is the likelihood, **P(C)** is the prior probability, and **P(X)** is the evidence. The class with the highest posterior probability is selected as the predicted class.

3. Bayesian Network

A Bayesian Network is a probabilistic graphical model that represents relationships between variables using a **Directed Acyclic Graph (DAG)**. Nodes represent variables and directed edges represent dependencies between them.

Example:

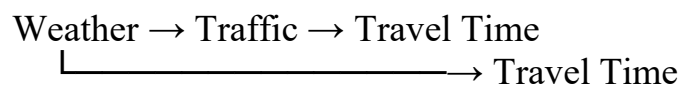


Figure: Example of Bayesian Network

Bayesian Networks are useful for representing uncertainty and calculating probabilities based on relationships between variables.

4. Data Preprocessing and Transformation

Before training the models, the dataset is cleaned by handling missing values, removing duplicates, and correcting inconsistent data. **Exploratory Data Analysis (EDA)** is performed to understand patterns, distributions, relationships, and outliers. The data is then transformed using encoding or scaling where required and divided into training and testing sets.

5. Model Training and Evaluation

The prepared training data is used to train the Naive Bayes and Bayesian Network models. The trained models are tested on unseen data and their performance is evaluated using **Accuracy, Precision, Recall, and F1-Score**. A confusion matrix can also be used to analyze correct and incorrect predictions. The models are compared to identify the better-performing classification model.

Conclusion

We successfully implemented the **Naive Bayes Classifier and Bayesian Network** for classification. The dataset was prepared through data preprocessing and transformation, followed by model training and performance evaluation. The models were compared using suitable classification metrics, demonstrating the application of Bayesian learning principles in solving classification problems.

Practical No.:	09
Practical Title :	Comparison of Classification Algorithms
Aim:	To compare the performance of different classification algorithms through model training and performance evaluation.
Objective:	To train different classification algorithms on a given dataset and analyze their performance using suitable evaluation metrics to identify the best-performing model.

Theory:**1. Classification**

Classification is a supervised Machine Learning technique used to predict categorical class labels. The model learns patterns from training data and assigns new observations to predefined classes.

2. Classification Algorithms

Different algorithms can be used for classification, such as Logistic Regression, Decision Tree, K-Nearest Neighbors (KNN), Support Vector Machine (SVM), and Naive Bayes. Each algorithm uses a different approach to learn the relationship between input features and the target class.

3. Model Training

The dataset is divided into training and testing sets. The training data is used to build the classification models, while the testing data is used to measure their performance on unseen data.

4. Performance Evaluation

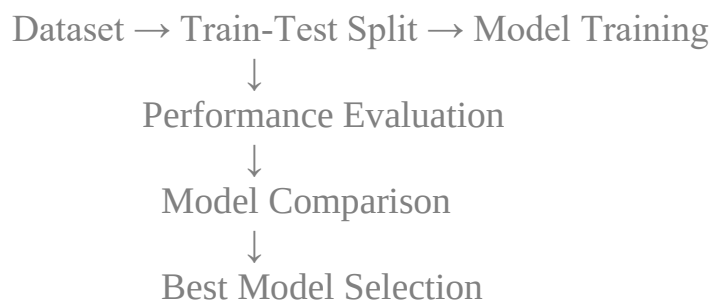
The trained models are evaluated using metrics such as Accuracy, Precision, Recall, and F1-Score. A confusion matrix can also be used to analyze correct and incorrect predictions.

$$\begin{aligned} \text{Accuracy} &= \frac{TP + TN}{TP + TN + FP + FN} \\ \text{Precision} &= \frac{TP}{TP + FP} \\ \text{Recall} &= \frac{TP}{TP + FN} \\ \text{F1-Score} &= \frac{2 \times \text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

5. Model Comparison

The performance of different classification algorithms is compared using the evaluation metrics. The model providing better performance on unseen data is selected as the most suitable model for the given classification problem.

Workflow:



Conclusion

We successfully trained and evaluated different classification algorithms using suitable performance metrics. The models were compared based on their evaluation results, allowing the best-performing classification algorithm to be identified for the given dataset.

Practical No.:	10
Practical Title :	Implementation of Support Vector Machine (SVM) Model
Aim:	To implement a Support Vector Machine (SVM) model by performing data cleaning, exploratory data analysis, data transformation, and dimensionality reduction.
Objective:	To understand and implement the Support Vector Machine algorithm by preparing and analyzing the dataset, transforming the features, reducing dimensionality, and applying SVM for effective classification.

Theory:

1. Support Vector Machine (SVM)

Support Vector Machine is a supervised Machine Learning algorithm mainly used for classification. It finds an optimal hyperplane that separates data points belonging to different classes. The closest data points to the hyperplane are called support vectors.

2. Hyperplane and Margin

The main objective of SVM is to find a hyperplane that separates classes with the maximum possible margin. For a linear SVM, the hyperplane is represented as:

$$\mathbf{w}^T \mathbf{x} + b = 0$$

where $\mathbf{w}$ is the weight vector, $\mathbf{x}$ is the input feature vector, and b is the bias. The margin between the classes is maximized to improve the model's ability to generalize to unseen data.

3. Data Cleaning

Data cleaning involves handling missing values, removing duplicate records, correcting inconsistent values, and dealing with unwanted or noisy data. Clean data helps improve the performance of the SVM model.

4. Exploratory Data Analysis (EDA)

EDA is performed to understand the dataset, feature distributions, relationships, correlations, and outliers. Histograms, box plots, scatter plots, and correlation heatmaps can be used for analysis.

5. Data Transformation

SVM is sensitive to the scale of features. Therefore, numerical features are commonly standardized or normalized before training. Categorical features can be converted into numerical form using encoding techniques.

6. Dimensionality Reduction

Dimensionality reduction reduces the number of features while retaining the most important information. Principal Component Analysis (PCA) is a commonly used technique for this purpose. It can reduce computational complexity, remove redundant information, and help visualize high-dimensional data.

The principal components are obtained from the covariance structure of the data, and the components with the highest variance are selected.

PCA workflow:

Original Features



Data Standardization



PCA



Reduced Features



SVM Model

7. Model Training

After preprocessing and dimensionality reduction, the transformed data is used to train the SVM model. The model learns the decision boundary that best separates the classes.

Conclusion

We successfully implemented the Support Vector Machine model by performing data cleaning, exploratory data analysis, data transformation, and dimensionality reduction. The SVM model uses an optimal separating hyperplane to classify data, while dimensionality reduction helps simplify the feature space. Thus, the practical demonstrated the application of SVM on prepared and reduced data.